BTS SNIR session 2023

BTS Systèmes Numériques
Option : IR
E 6-2 – PROJET TECHNIQUE

Dossier de présentation et de validation du projet *(consignes et contenus)*

Groupement académique : Créteil Paris Versailles	**Session : 2023**
Lycée :JOSEPH GAILLARD	
Ville : FORT-DE-FRANCE	
N° du projet :	**Nom du projet : Réseau Télémétrique de Mesures en Temps Réel. (R.T.M.T.R n°1) (Mesures sismiques)**

Projet nouveau	Oui x	Non			Projet interne	Oui	Non x
					Statut des étudiants	Formation initiale x	Apprentissage
Spécialité des étudiants	EC	IR x	Mixte		Nombre d'étudiants : 4		
Professeurs responsables :							

I_1_Présentation du projet

Il s'agit de réaliser un réseau télémétrique de mesures en temps réel de stations qui fourniront un flot de données sismiques et la position du lieu d'acquisition des mesures. Le dispositif consiste à offrir un observatoire d'activités sismiques en un point ou une zone géographique déterminée. Ce système sera capable de s'intégrer au sein d'un réseau déjà existant de stations sismiques, réalisant ainsi la couverture d'une zone qui s'étendra en fonction du nombre de stations déployées.

I_2_Expression du besoin

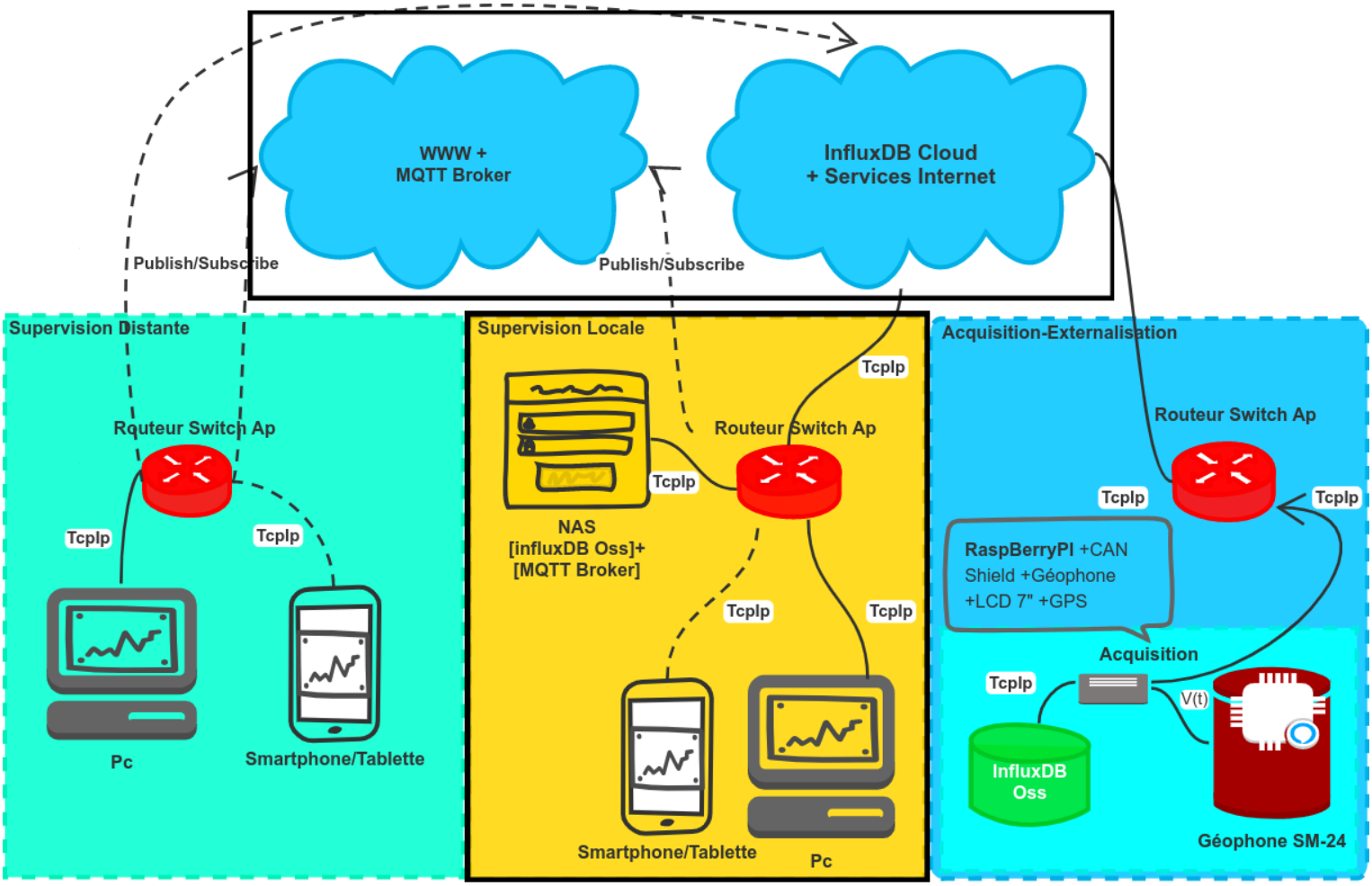
L ‘entreprise Vision Informatique souhaite réaliser un réseau télémétrique de mesures en temps réel de stations connectées qui fourniront un flot de données sismiques et de position. L’idée est d’offrir un observatoire d'activités sismiques pour un point donné (Gps) ou sur une zone. En collaboration avec la collectivité territoriale de Martinique , l’idée est de sensibiliser les élèves de collèges sur les risques majeurs qui sont nombreux dans notre zone géographique (risques : tsunamis, glissements de terrain, volcaniques, lahars, séismes, etc..).

Les stations embarqueront une base de données InFluxDB_Oss pour enregistrer les relevés. *Les TSDB (time series databases), ou bases de données de séries chronologiques, sont des systèmes logiciels optimisés pour trier et organiser des informations mesurées de manière temporelle. Une série chronologique est une série de points de données regroupés à des intervalles successifs et par ordre d'arrivée. Dans le jargon associé aux TSDB, un point de données correspond à une paire heure-valeur ou clé-valeur.*

Les stations devront transmettre périodiquement l'ensemble de leurs relevés et leur position au serveur installé dans les services techniques de la collectivité. L'application du serveur devra présenter l'ensemble des données sous forme de graphes (sismogrammes), afin d'observer l’activité sismique en chaque point de relevés de une ou plusieurs stations. Il sera aussi possible à partir de ces relevés de déterminer les foyers, les épicentres, les magnitudes.

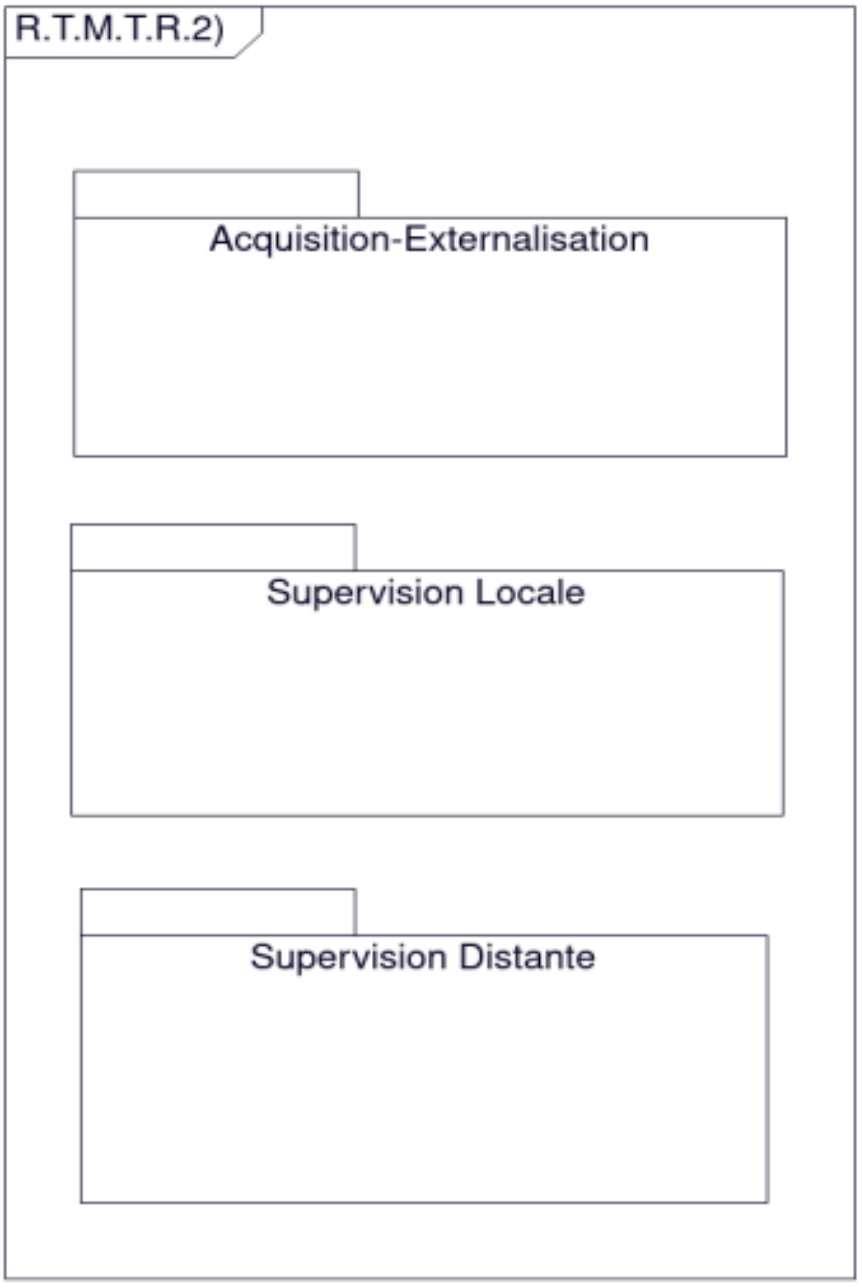
En cas d'interruption de communication avec la supervision, chaque station sera capable d’enregistrer les périodes d'observation d'activités et pourra les transmettre ultérieurement au serveur, une fois la communication rétablie. A des fins pédagogiques il sera aussi possible d’observer soit l'activité sismique temps réel, soit d’observer des activités sismique sur une période antérieure (minutes, heures, jours, mois années) grâce à la *base de données de séries chronologiques* embarquée sur chaque station.

I_3_Synoptique



II_SPÉCIFICATION

Le système sera découpé en 3 sous-systèmes



Le travail que l'on me demande de réaliser en tant qu'étudiant n°3 est mentionné en page 9 du cahier des charges. Je dois réaliser le sous-système local.

II_1_SPÉCIFICATION

Supervision Locale:

A)Une fonctionnalité fournie par le constructeur permet la réplication automatique de la base de données InfluxDB_Cloud_Raspberry avec la base de données InfluxDB_Oss installée sur le serveur NAS.

Mettre en œuvre la réplication

B)Il faut pouvoir répondre au demande de la supervision distante , on doit mettre en place un broker Mqtt en ligne (Mosquitto).

définir un topic de souscription et topic de publication

Mettre en oeuvre un service Mqtt Broker en ligne

la supervision distante publie une demande de période d'observation (topic demande)

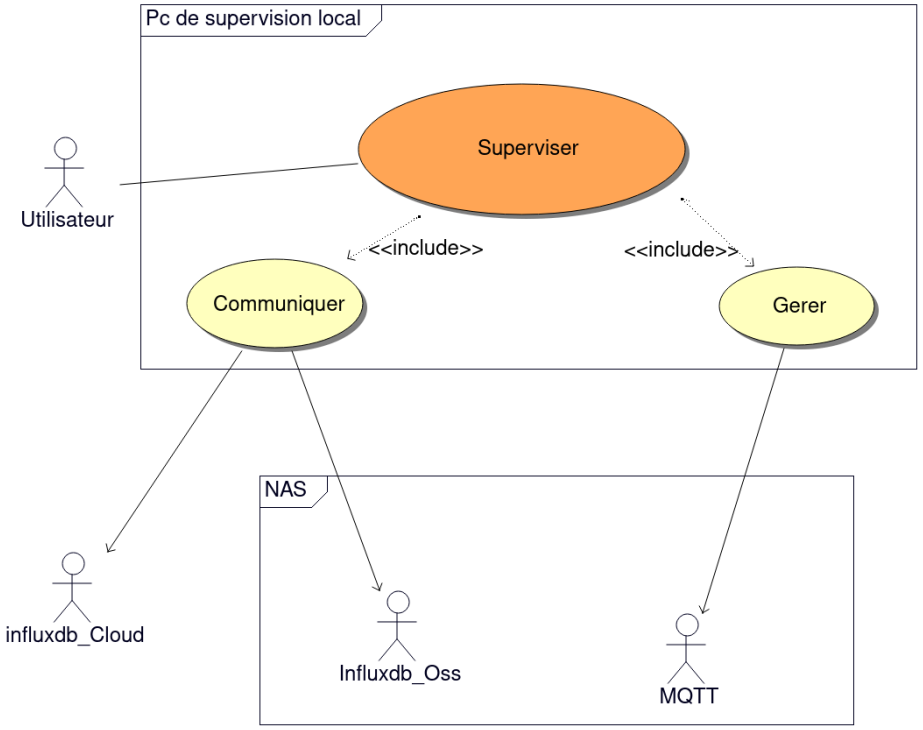
la supervision locale souscrit à cette demande de période d'observation

la supervision récupère dans la BD les données sur cette période et les publie (topic reponse)

C) **Au niveau du Pc de supervision:** La supervision s'effectue depuis un PC à partir d'une application dédiée (Qt5). L'application procède par des requêtes (période d'observation) afin de visualiser les données sismiques.

D) **Au niveau du Smartphone/Tablette:** Il faudra pour le smartphone procéder des requêtes sur la base de données InfluxDB_Oss du NAS et afficher les résultats sur une Interface homme machine native d'Android (Affichage d'un intervalle d'observation, ou affichage en continu).

1 Diagramme des cas d'utilisation



2 Description des acteurs

- ➢ Utilisateur
- ➢ Pc
- ➢ InfluxDB_Cloud
- ➢ InfluxDB_Oss

Acteur	L'utilisateur depuis le pc de supervision utilise un navigateur pour consulter les données d'une station de mesure sismiques (en un point) ou de plusieurs stations (sur une zone).
Utilisateur	L'utilisateur consulte les données depuis son Pc via le navigateur et profite des fonctionnalités intégrées de présentations formatées de données que lui offre InfluxDb_Oss. Une autre fonctionnalité permet par la détermination de seuils afin de générer des alertes, en fonction des données qui alimentent la base de données InfluxDB_Oss. Il peut aussi utiliser son smartphone/tablette pour visualiser les données via une application dédiée à réaliser.
Pc	L'utilisateur depuis le pc de supervision utilise un navigateur pour consulter les données d'une station de mesure sismiques (en un point) ou de plusieurs stations (sur une zone).

InfluxDB_Cloud	Les relevés sismiques effectués par le sous-système Acquisition-Externalisation sont enregistrés dans cette base de données.
InfluxDB_Oss	InfluxDB_Cloud alimente par réplication influxDB_Oss. Le serveur de Base de Données Time Series InfluxDB_Oss installé sur le serveur NAS intègre des outils de présentation de graphe depuis un dashboard et accessible depuis un Browser (Affichage d'un intervalle d'observation, ou affichage en continu).

3 Description des cas d'utilisation

Cas d'utilisation	Rôle
Superviser	**Au niveau Pc :** Utilisation des fonctionnalités intégrées dans InfluxDb_Oss pour la présentation des données et des alertes. **Au niveau Smartphone/Tablette :** visualiser les données via une application dédiée.
Gérer	Fonctionnalité à utiliser par l'application Android. L'application procédera par des requêtes sur la base de données et présentera les résultats sur une interface dédiée.

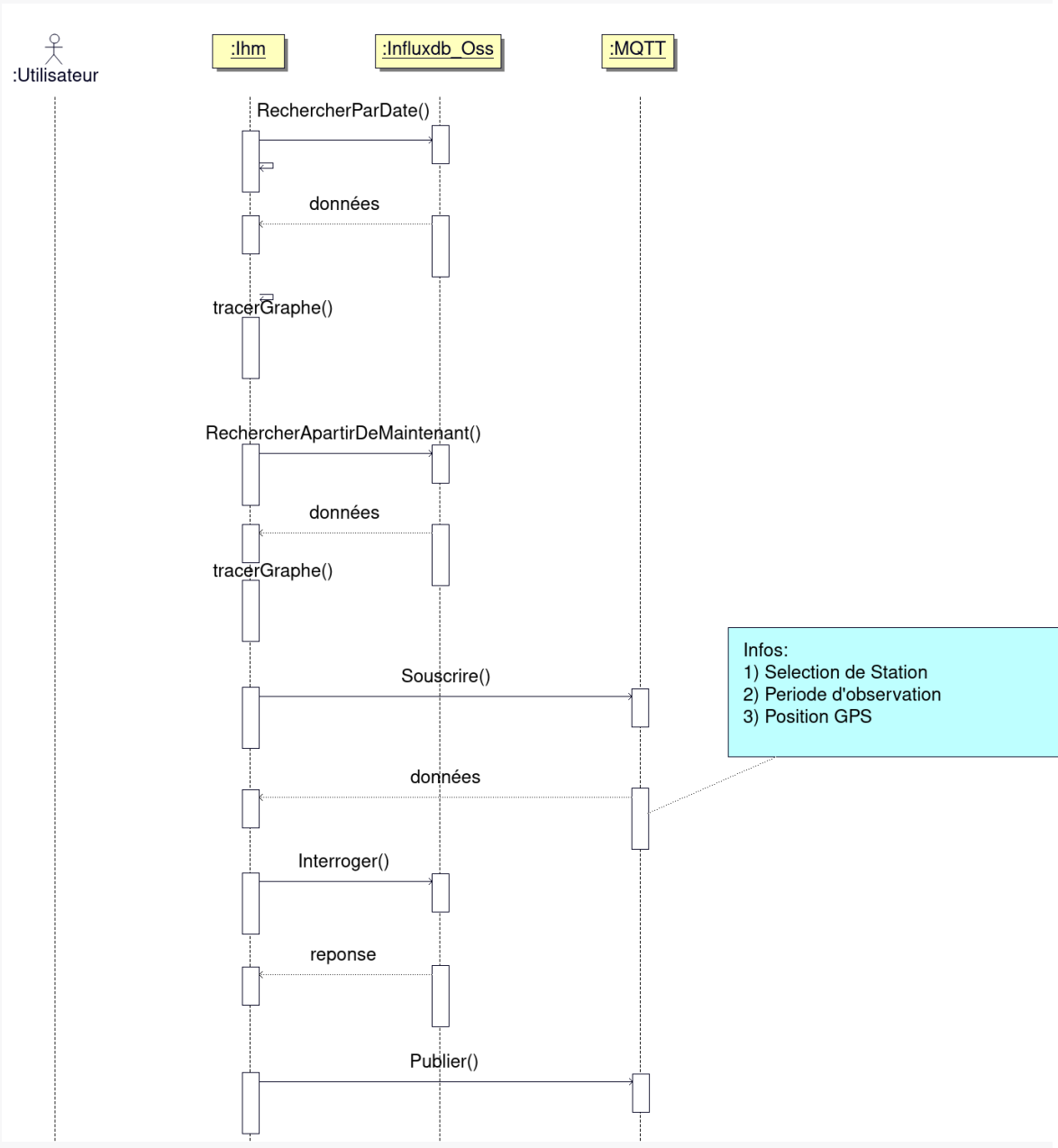
Travail demandé dans le cahier des charges :

Coder un module logiciel graphique pour Android pour l'affichage des mesures (courbes, jauges, Etc..). La base de données InfluxDB_Oss fournit (requêtes) les mesures à présenter sur l'interface graphique.

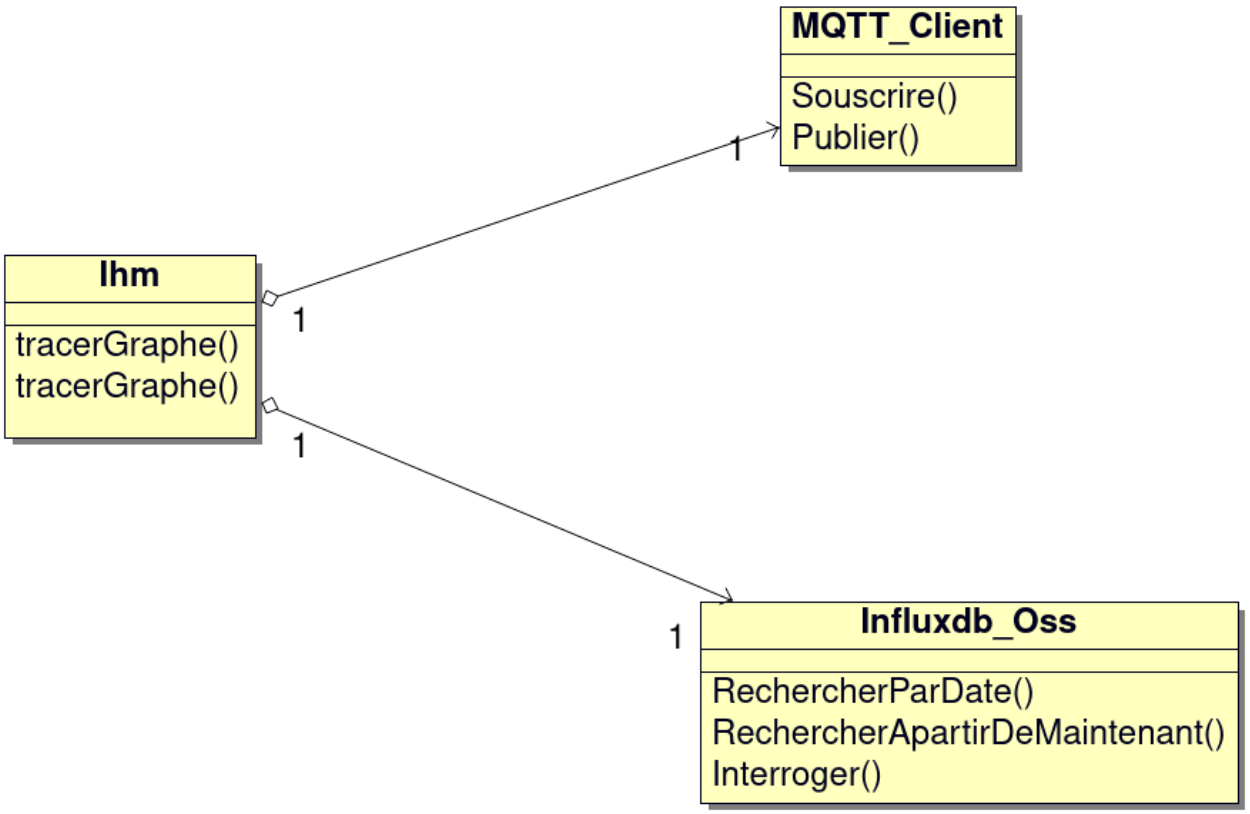
plan détaillé : Coder un module qt5 logiciel graphique pour Android pour l'affichage des mesures (courbes, jauges, Etc..). La base de données InfluxDB_Oss fournit (requêtes) les mesures à présenter sur l'interface graphique.

1. Préparation de l'environnement de développement :
    - Installer et configurer l'environnement de développement Qt5 sur votre ordinateur.
    - S'assurer que l'environnement de développement est compatible avec la cible Android et que tous les outils nécessaires sont disponibles (émulateur Android, kits de développement logiciel (SDK), etc.).
2. Création du projet Qt5 :
    - Ouvrir l'environnement de développement Qt5 et créer un nouveau projet de type "Application graphique Qt pour Android".
    - Configurer les paramètres du projet (nom, répertoire, etc.) et créer l'interface graphique de base (fenêtre principale, barre de menus, barre d'outils, etc.).
3. Connexion à la base de données InfluxDB_Oss :
    - Importer les modules de connexion à la base de données InfluxDB_Oss dans votre projet Qt5.
    - Écrire le code nécessaire pour établir une connexion à la base de données et exécuter les requêtes de récupération de données.
4. Affichage des mesures :
    - Créer les widgets graphiques (courbes, jauges, etc.) qui permettront d'afficher les mesures récupérées de la base de données.
    - Écrire le code pour mettre à jour les widgets en fonction des données récupérées à partir de la base de données.
5. Test et débogage :
    - Exécuter le projet sur l'émulateur Android ou sur un appareil physique pour vérifier que tout fonctionne correctement.
    - Identifier et corriger toutes les erreurs ou problèmes rencontrés pendant les tests.
6. Génération de l'application finale :
    - Utiliser l'outil de génération de l'environnement de développement Qt5 pour créer un fichier d'installation de l'application (fichier APK pour Android).
    - Tester l'application générée sur l'émulateur ou sur un appareil physique pour s'assurer qu'elle fonctionne correctement.

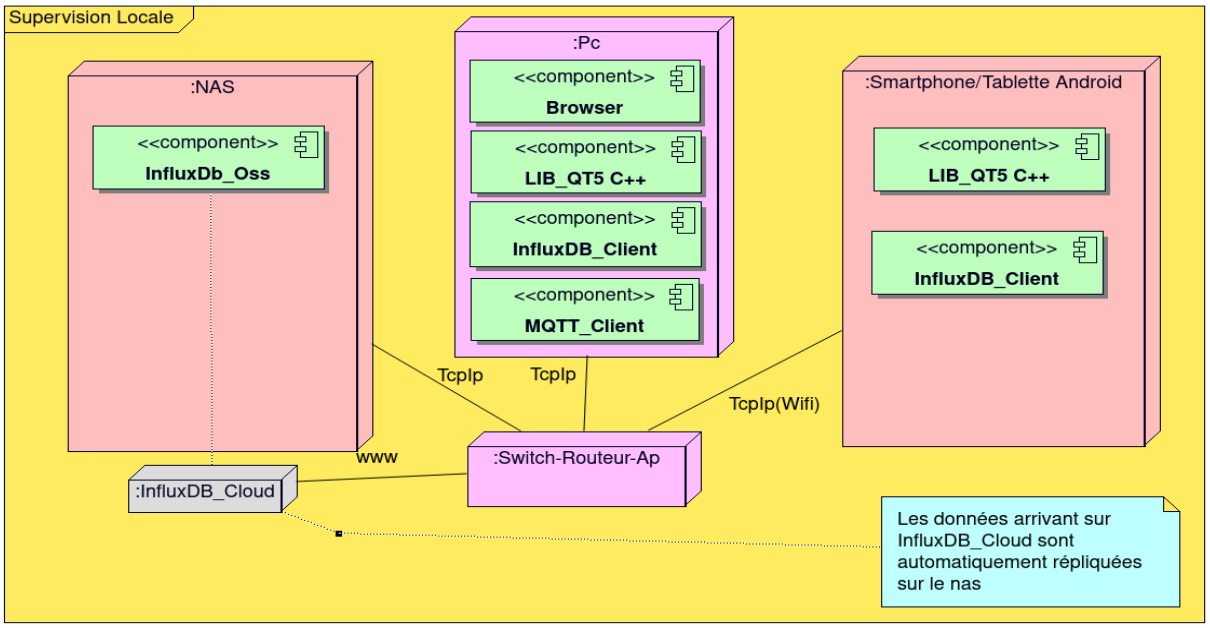
1 Diagramme de séquence



2 Diagramme de classe



3 Diagramme de classe d'analyse



A ce stade, j'ai fait émergé le diagramme de classe d'analyse. Pour réaliser le sous système, je dois prendre en compte les contraintes matérielles et logicielles.

4 Les contraintes matérielles et logicielles.

➢ CHOIX MATÉRIELS.

- 1 Pc
- 1 tablette ou téléphone Android
- 1 nas Synology
- 1 switch

➢ CHOIX LOGICIELS.

- pour le PC : Linux , Framework Qt 5
- pour la tablette: Framework Qt5
- Utilisation de InfluxDb Cloud
- Pour nas influxDB_Oss et MQtt Broker

5 Mise en oeuvre de la conception

5.1 Rappel du travail demandé:

Coder un module logiciel graphique pour Android pour l'affichage des mesures (courbes, jauges, Etc..). La base de données InfluxDB_Oss fournit (requêtes) les mesures à présenter sur l'interface graphique.

Je propose un plan détaillé le plan détaillé suivant:

2. Préparation de l'environnement de développement :
    - Installer et configurer l'environnement de développement Qt5 sur votre ordinateur.
    - S'assurer que l'environnement de développement est compatible avec la cible Android et que tous les outils nécessaires sont disponibles (émulateur Android, kits de développement logiciel (SDK), etc.).
3. Création du projet Qt5 :
    - Ouvrir l'environnement de développement Qt5 et créer un nouveau projet de type "Application graphique Qt pour Android".
    - Faire une ihm intermédiaire pour vérifier le bon fonctionnement du projet
    - Configurer les paramètres du projet (nom, répertoire, etc.) et créer l'interface graphique de base (fenêtre principale, barre de menus, barre d'outils, etc.).
4. Connexion à la base de données InfluxDB_Oss :
    - Importer les modules de connexion à la base de données InfluxDB_Oss dans votre projet Qt5.
    - Écrire le code nécessaire pour établir une connexion à la base de données et exécuter les requêtes de récupération de données.
5. Affichage des mesures :
    - Créer les widgets graphiques (courbes, jauges, etc.) qui permettent d'afficher les mesures récupérées de la base de données.
    - Écrire le code pour mettre à jour les widgets en fonction des données récupérées à partir de la base de données.
6. Test et débogage :
    - Exécuter le projet sur l'émulateur Android ou sur un appareil physique pour vérifier que tout fonctionne correctement.
    - Identifier et corriger toutes les erreurs ou problèmes rencontrés pendant les tests.
7. Génération de l'application finale :
    - Utiliser l'outil de génération de l'environnement de développement Qt5 pour créer un fichier d'installation de l'application (fichier APK pour Android).
    - Tester l'application générée sur l'émulateur ou sur un appareil physique pour s'assurer qu'elle fonctionne correctement.

Pour pouvoir réaliser ce projet, il nous faut ajouter le fichier influxdb.hpp qui nous permettra de gérer la connexion au serveur influxdb.
Principe InfluxDB V2
Afin de réaliser la programmation de ma partie, il faut tout d'abord mettre au clair les différents objets utilisés :

Le bucket : Récipient principal contenant l'ensemble de nos données

Le Filtre _measurement : Élément de sélection principal pour différencier les stations

Le filtre _field : Contient les valeurs affichable graphiquement

J'ai utilisé ces objets de la façon suivante :

bucket " " → _measurement " " → _field "mesure" & "position"

Afin de vérifier le bon fonctionnement de cette méthode, nous allons faire un test d'enregistrement avec des variables fixes



5.2 PROGRAMMATION

Tout d'abord, je me suis concentré sur l'enregistrement de données dans la base que l'équipe à préalablement créée sur influxdb avec l'adresse à l'adresse ip suivante : 192.168.1.200:49153

- Structure de la base de données

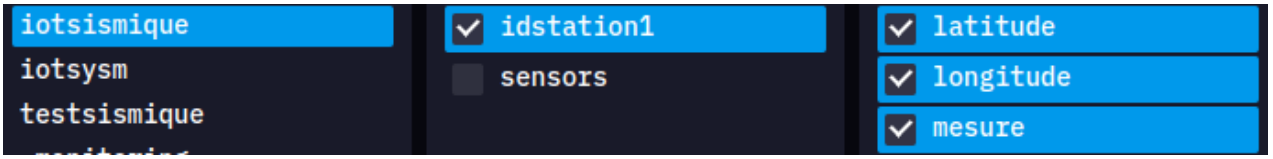
  bucket -> measurement -> field

  InfluxDB_Oss définit trois zones

  1. bucket : choix du nom de la base de données
  2. measurement : choix du nom du measurement
  3. field : choix des noms des mesures effectives

- Conception de la base de données

  bucket -> measurement -> field
  iotsismique -> **idstation1** -> **mesure**
  latitude
  longitude



Je l'ai utilisé au départ dans le but d'entreprendre des tests d'enregistrement et de lecture de données autonomes.
Pour ne pas être pénalisé par mes coéquipiers, j'ai créé une fonction qui enregistre des valeurs tapées dans l'ihm dans la base de données comme ceci :

```
void Widget::on_enregistrer_clicked()
{

    QString IdStat = "idstation1";
        int a = ui->lvaluemesure->text().toInt();
        int b = ui->lvalueposition->text().toInt();
        int c = ui->longitude->text().toInt();
 influxdb_Oss::server_info si("192.168.1.200",
49153, "btssnir",
jeton masque
 "iotsismique");
influxdb_Oss::builder()
        .meas(IdStat.toStdString())
         // .tag("station", "Station 1")
          .field("mesures", a)
          .field("latitude", b)
          .field("longitude",c)
           //.timestamp(1512722735522840439)
            .post_http(si);
```

Elle utilise l'adresse ip, le port, l'organisation, le token créé, et le bucket également créé. Pour créer un measurement idstation1 puis des fields mesures, latitude, et longitude.





Ensuite, je me suis occupé de créer une fonction capable de lire les trames de données enregistrées d'abord pendant une durée déterminée.

Une trame se présente de cette façon :
,result,table,_start,_stop,_time,_value,_field,_measurement
Et est représenté en c++ de la façon suivante :
"from(bucket: \\\"iotsismique\\\")|> range(start: -1h)|>filter(fn: (r)=>r[\\\"_measurement\\\"] == \\\"idstation1\\\")|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\"mesure\\\")")

```
void Widget::on_afficherGraphe_clicked()
{



    string s1="from(bucket: \\\"";
    string sbucket=ui->lbucket->text().toStdString()+"\\\")";
```

```
string range= "|> range(start: -1m)";
string range1;
if(ui->radiosecondes->isChecked())
range1= "|> range(start: -"+ui->ltime->text().toStdString()+"s)";
if(ui->radiominutes->isChecked())
range1= "|> range(start: -"+ui->ltime->text().toStdString()+"m)";
if(ui->radioheures->isChecked())
range1= "|> range(start: -"+ui->ltime->text().toStdString()+"h)";
```

Cette partie sert à donner une durée maximale pour rechercher les données existantes dans la base.
Au départ, je me suis concentré sur la valeur de “mesures”

```
string filter1="|>filter(fn: (r)=>r[\\\"_measurement\\\"] == \\\""+ui->lmeasurement->text().toStdString()+"\\\")";
string filter2="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lfield->text().toStdString()+"\\\")";
```

Afin d’enregistrer dans la base il me faut me connecter à celle-ci :

```
servdb_=new server_info ("192.168.1.200",
49153,
"btssnir",
jeton masque
"iotsismique");
```

```
string resp;
string query = s1+sbucket+range1+filter1+filter2;
qDebug() << query.c_str();
flux_query(resp, query, *servdb_);
```

puis j’ai tout additionné dans une variable qui est devenue mon paquet de trames.

```
,_result,0,2023-03-21T15:04:28.972086963Z,2023-03-27T15:04:28.972086963Z,2023-03-27T14:43:33.664247595Z,4,mesure,idstation1
,_result,0,2023-03-21T15:04:28.972086963Z,2023-03-27T15:04:28.972086963Z,2023-03-27T14:43:35.12851597Z,-5,mesure,idstation1
```

```
QByteArray reponse = resp.c_str();
qDebug() << reponse;
qDebug() <<reponse.count("_result");


spltCrLf_ = reponse.split(0x0A);
```

Ici j’ai affiché le nombre de trames enregistrées dans cette période puis les ai séparées par ligne.

```
nbEchantillons = reponse.count("_result");
for(int i = 0; i < nbEchantillons; i++)
{
    ExtraireValueDate(spltCrLf_[i]);

}
```

Et ici j'utilise le nombre d'itérations de mes trames comme compteur pour le nombre de fois ou je vais répéter la séparation

```
spltVirgule_ = Trame.split(',');  //  Sépare la trame par champs avec une virgule comme moyen d'opération
QByteArray Date = spltVirgule_[5];
spltDate_ = Date.split('T');       // Sépare la 5eme partie de la trame avec le T en moyen d'opération
spltjma_ = Date.split('T');        // Sépare la 5eme partie de la trame avec le T en moyen d'opération
spltjma_.pop_back();               // Enlève la partie après le T pour n'en garder que le début
spltDate_.pop_front();             // Enlève la partie avant le T pour n'en garder que la fin
// qDebug() << spltjma_;           // Affiche ce qu'il reste avant le T
// qDebug() << spltDate_;          // Affiche ce qu'il reste après le T
QByteArray Heure = spltDate_[0];   // Garde la partie hh:mm:ss.z (heures minutes secondes)
spltHeure_ = Heure.split('.');     // Sépare la trame avec le . en moyen d'opération
spltHeure_.pop_back();             // Enlève la partie après le .
```

Une fois cette partie réalisée, toujours avec les valeurs que j'ai enregistrées à la main, j'ai créé un moyen de rechercher par dates les valeurs



Pour ce faire, j'ai créé une fonction pour afficher par période

```
void Widget::on_afficherpardate_clicked()
{
    QDateTime dateTimetStart = ui->DateDebut->dateTime();
    dateTimetStart.setTimeSpec(Qt::UTC);
    unsigned int utstart= dateTimetStart.toTime_t();

    QDateTime dateTimeStop = ui->DateFin->dateTime();
    dateTimeStop.setTimeSpec(Qt::UTC);
    unsigned int utstop = dateTimeStop.toTime_t();

    QString stringDate1= QString::number(utstart);
    QString stringDate2= QString::number(utstop);
```

En utilisant les objets de date montrés précédemment.

```
string s1="from(bucket: \\\"";
string sbucket=ui->lbucket->text().toStdString()+"\\\")";

string range1= "|> range(start: -5m)";
string range2= "|> range(start:  2022-11-16T05:30:00Z, stop: 2022-11-16T12:00:00Z)";
string range3= "|> range(start:  1668470400, stop: 1672084800)";
string range4= "|> range(start: "+stringDate1.toStdString()+", stop: "+stringDate2.toStdString()+")";

string filter1="|>filter(fn: (r)=>r[\\\"_measurement\\\"] == \\\""+ui->lmeasurement->text().toStdString()+"\\\")";
string filter2="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lfield->text().toStdString()+"\\\")";
string filter3="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lfield2->text().toStdString()+"\\\")";
string filter4="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lfield3->text().toStdString()+"\\\")";
```

```
string resp;
string query = s1+sbucket+range4+filter1+filter3;
```

Puis j'initialise la forme que prendra ma trame

```
flux_query(resp, query, *servdb_);


qDebug() << resp.c_str();
QByteArray rep = resp.c_str();
qDebug() << rep.count("_result");
nbEchantillons = rep.toInt();
```

Et ici, j'affiche en sortie d'application le paquet de trames ainsi que le nombre d'occurrences du mot "_result" afin de savoir le nombre de trames présente dans ma période

Une fois que mes coéquipiers ont réalisé leurs parties respectives (étudiant 1 et 2) j'ai pu perfectionner la version finale du projet.
Elle consiste en un regroupement des 2 étapes de recherche faites précédemment en une seule qui va afficher, à partir d'une date donnée la minute suivante sous forme de graphique.
Pour effectuer cette étape, j'ai utilisé la même base que pour les 2 fonctions précédentes en y apportant quelques modifications mineures

```
void Widget::on_afficherpardate_clicked()
{
   ui->groupBox->show();
     ui->goss->hide();  ui->gmqtt->hide();


      startDate = ui->DateDebut->dateTime();
      qDebug() << QString (ui->DateDebut->dateTime().toString()).toUtf8();
      startDate.setTimeSpec(Qt::UTC);
      unsigned int utstart= startDate.toTime_t();
      ui->DateDebut_2->setText(startDate.toString());


      QString stringDate1= QString::number(utstart);
      QString stringDate2= QString::number(utstart+ui->ldureeobs->text().toUInt());


      string s1="from(bucket: \\\"";
      string sbucket=ui->lbucket->text().toStdString()+"\\\")";

      string range1= "|> range(start: -5m)";
      string range2= "|> range(start:  2022-11-16T05:30:00Z, stop: 2022-11-16T12:00:00Z)";
      string range3= "|> range(start:  1668470400, stop: 1672084800)";
      string range4= "|> range(start: "+stringDate1.toStdString()+
                     ", stop: "+stringDate2.toStdString()+")";

string filter1="|>filter(fn: (r)=>r[\\\"_measurement\\\"] == \\\""+ui->measurement->text().toStdString()+"\\\")";
string filter2="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+string("mesure")+"\\\")";
string filter3="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lbllat->text().toStdString()+"\\\")";
string filter4="|>filter(fn: (r)=>r[\\\"_field\\\"] == \\\""+ui->lbllong->text().toStdString()+"\\\")";

string resp;
string query = s1+sbucket+range4+filter1+filter2;//+filter2+filter4

qDebug() << query.c_str();


flux_query(resp, query, *servdb_);
QByteArray reponse = resp.c_str();
qDebug() <<reponse;
qDebug() <<"-- "<<reponse.count("_result");

///-- @Extractions des informations uiles --
spltCrLf_ = reponse.split(0x0A);
spltCrLf_.pop_front();
//on enleve la ligne d'information sur les trames de reponse
nbEchantillons = reponse.count("_result");
ExtraireValueDate(spltCrLf_,nbEchantillons);
 ui->plot_4->setUnits("     Echantillonage = "+ui->ltauxech->text()+" ms");
TraceGraphe( ListHeure, ListValeur);
RecupererLongitude();
RecupererLatitude();
QString a = QString::number(longitude);
QString b = QString::number(latitude);
ui->longitude->setText(a);
ui->lattitude->setText(b);
ui->position->setText("Position : ( "+a+" , "+b+" )");
```

Cette fonction sert à utiliser la requête influxdb pour les données avec les paramètres personnalisés, qu'on additionne dans une seule variable qu'on envoie avec flux_query
Enfin, on sépare la réponse (un paquet de trames), on enlève la 1ère réponse (pop_front), on compte le nombre de données reçues sur l'intervalle et enfin on extrait la valeur avec la fonction ExtraireValueDate.
Avec ces valeurs on peut tracer le graphique avec la fonction TracerGraphe. Petit ajout, la position qui est directement envoyée sur le fond du graphique.

Résultat :



Ensuite, j'ai dû m'attaquer à la partie du broker mqtt afin de satisfaire l'étudiant 4, la supervision distante.
Pour réaliser cette partie je devais renseigner l'adresse ip mqtt, le port, le topic de publication ainsi que celui de souscription.

Ainsi que d'effectuer la connexion

```
mqtt_sub->Connexion(ui->lipbroker->text(),ui->lportbroker->text().toInt(),ui->ltopicsub->text());
mqtt_pub->Connexion(ui->lipbroker->text(),ui->lportbroker->text().toInt(),ui->ltopicpub->text());
connect(mqtt_sub,SIGNAL(onReceived(QByteArray)),this,SLOT(RecevoirMessage(QByteArray)));
```

Il faut, après avoir reçu la date de la supervision distante, envoyer la requête sur la base de donnée influxdb
Et ensuite envoyer ces données automatiquement sur le topic de publication

```
mqtt_pub->Publish(6,ui->ltopicpub->text(),(QString::number(ListValeur.first())).toUtf8());
```

Ce projet à été pensé pour être utilisable sur tablette en la branchant via câble usb sur l'ordinateur utilisant ce programme.
Pour qu'elle soit compatible, il faut rentrer une commande, montrée ci-dessous, afin de reconnaître l'appareil.
Ne pas oublier d'accepter le débogage USB pour que la liaison fonctionne

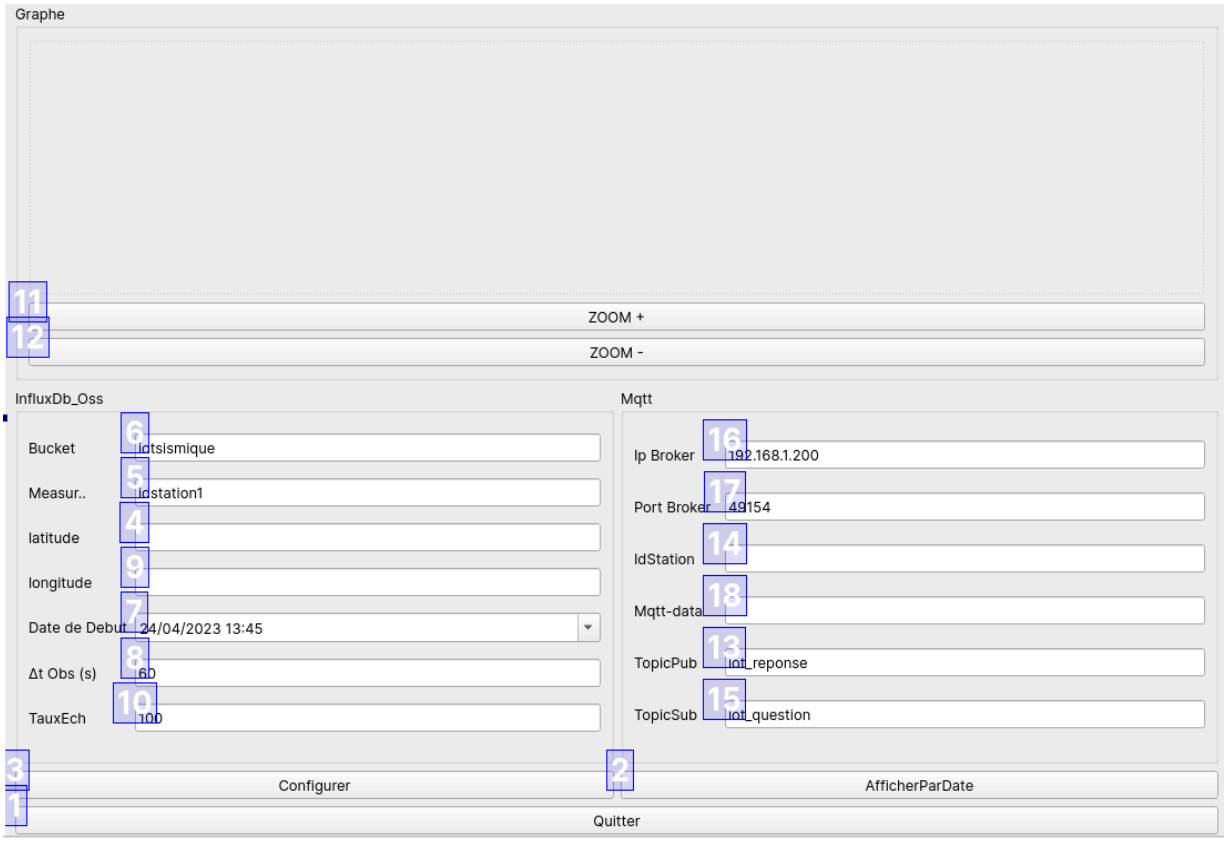
```
e11g1@Poste11:~$ adb devices
List of devices attached
adb server version (41) doesn't match this client (39); killing...
* daemon started successfully
G4NKFPC022002SY device

e11g1@Poste11:~$
```

L'ihm étant adaptative, elle prend donc automatiquement la totalité de l'écran peut importe le support utilisé

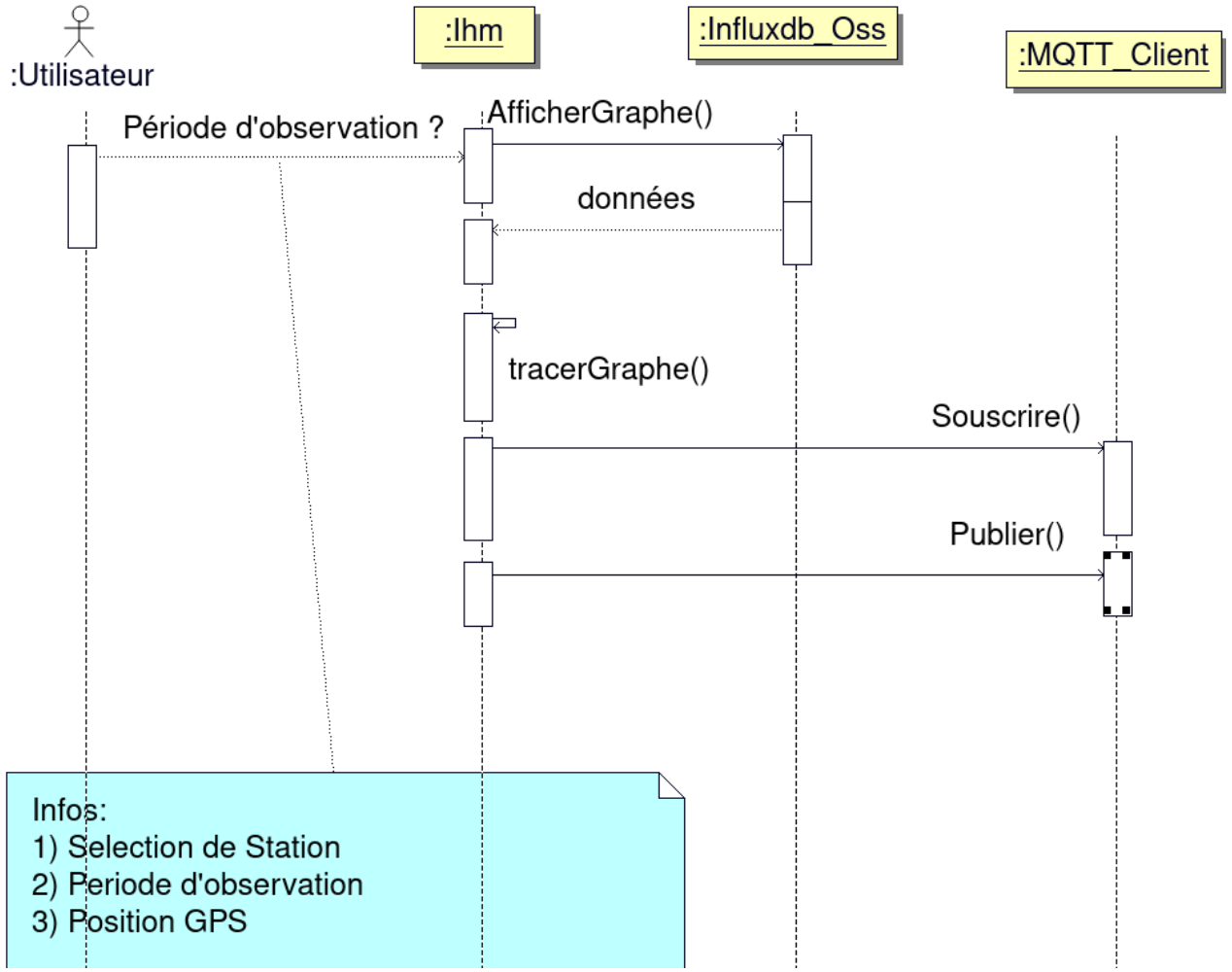
5.3 **En résumé**

Voici l'ihm finale de mon projet

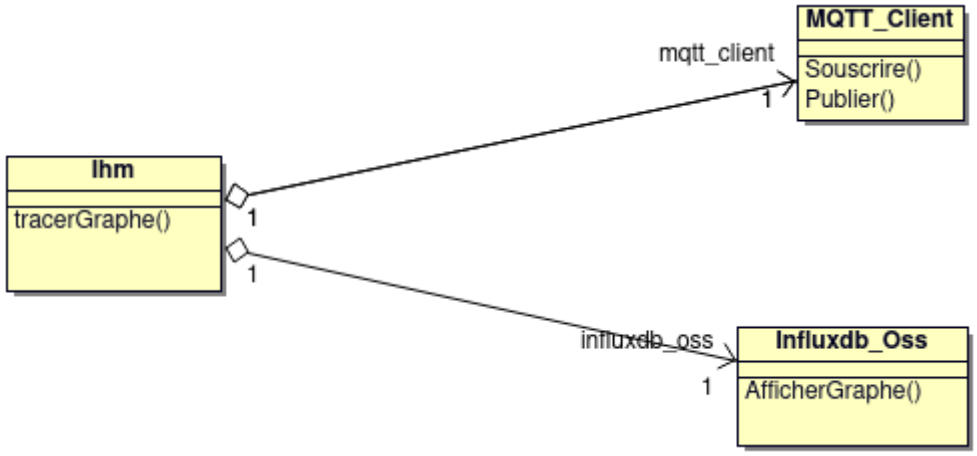


1.Bouton pour fermer l'IHM
2.Bouton permettant l'affichage de la courbe
3.Bouton permettant de revenir sur cette interface après avoir affiché la courbe
4.Champ d'enregistrement de latitude
5.Champ de measurement de la base
6.Nom de la base
7.Date de la période d'observation
8.Renseignement de la durée d'observation
9.Champ de longitude
10.Taux d'échantillonnage
11.Bouton de zoom avant du graphique
12.Bouton de zoom arrière du graphique
13.Topic de publication MQTT
14.Id de la station
15.Topic de souscription MQTT
16.Adresse ip du broker MQTT
17.Port du broker MQTT
18.Récupération de la donnée envoyée (date période)

Voici le diagramme de séquence final de mon projet :



Et voici le diagramme de classe final de mon projet :



6 Conclusion

Ce projet nous a permis de découvrir les différentes façons d'accéder à une base de données pour y lire les données et les utiliser dans plusieurs activités variées.

Il m'a personnellement appris à créer un graphique à partir de données recueillies, à séparer une trame de plusieurs parties, à gérer le dialogue sur le service mqtt (publication et souscription) et enfin à consolider mes acquis en c++ grâce aux nombreuses fonctions utilisées.

Au niveau de la gestion du projet en équipe, nous avons réussi à bien nous répartir les tâches avec moi comme cœur du projet afin de réaliser nos objectifs dans les temps et l'ambiance générale du groupe était très bonne.